Ex.No:2 Developing and Deploying a Simple Chaincode

Aim

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Procedure Step 1: Navigate to the Test Network

Open the terminal and move to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

Start the blockchain network with Certificate Authorities.

```
./network.sh up createChannel -ca
```

Expected Output

Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Network Started Successfully

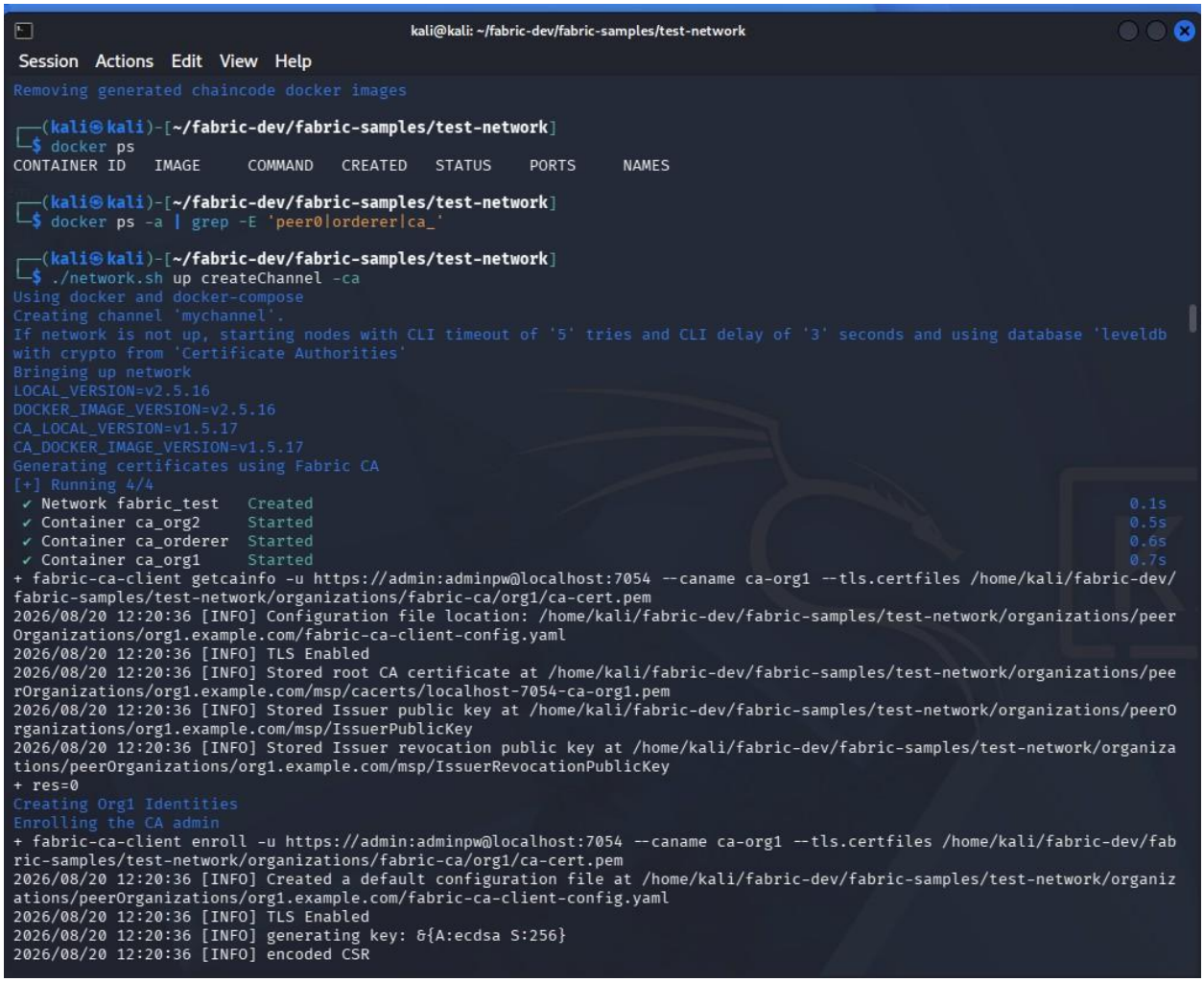

Figure 2.1: Fabric network and channel creation

Step 3: Deploy the JavaScript Chaincode

Deploy the Asset Transfer Basic chaincode.

./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript

Expected Output

Packaging chaincode
Installing chaincode
Approving chaincode
Committing chaincode
Chaincode committed successfully

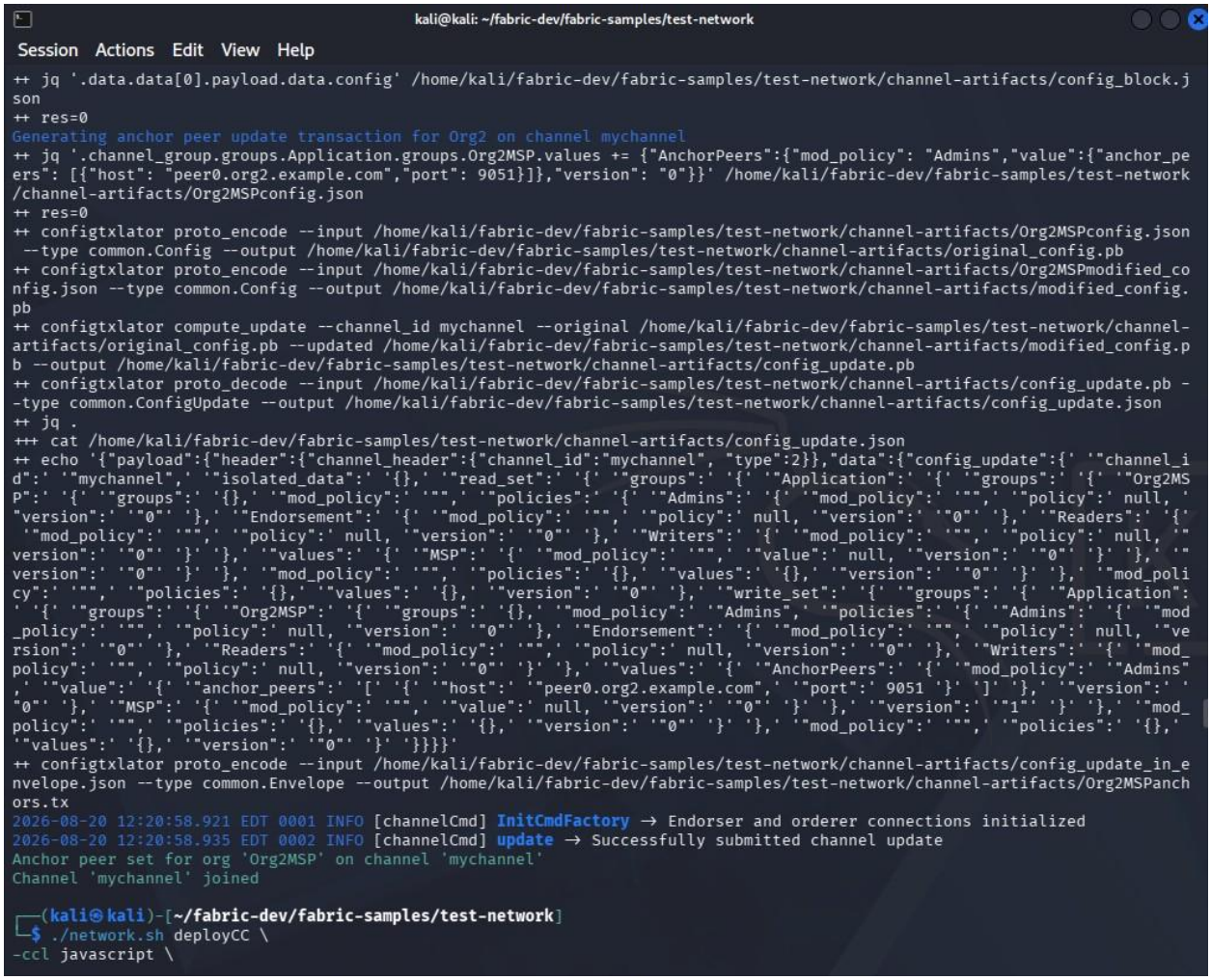

Figure 2.2: Chaincode deployment

Step 4: Verify Installed Chaincode

Check whether the chaincode is installed.

peer lifecycle chaincode queryinstalled

Expected Output

Package ID:
basic_1.0
Version:1.0

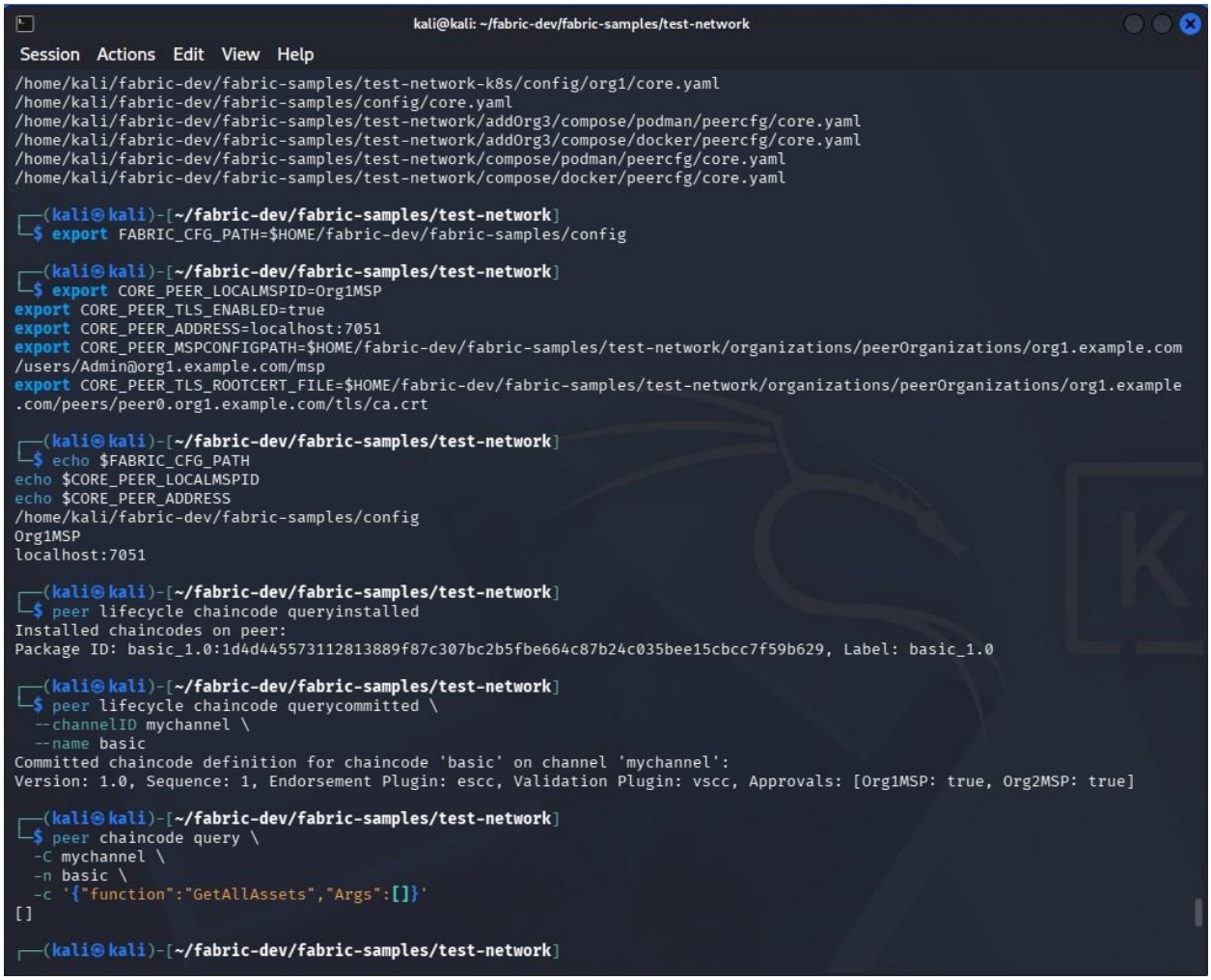

Figure 2.3: Verification of installed and committed chaincode

Step 5: Invoke the Chaincode

Initialize the ledger.

peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'

Expected Output

Chaincode invoke successful
Transaction committed

Step 6: Query the Ledger

Retrieve all assets stored in the blockchain.

peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'

Expected Output

[
{
"ID":"asset1",
"Color":"blue",
"Size":5,
"Owner":"Tomoko",
"AppraisedValue":300
},
{
"ID":"asset2",
"Color":"red",
"Size":5,

"Owner":"Brad",
"AppraisedValue":400
}
]

Step 7: Create a New Asset

peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'

Expected Output

Asset asset10 created successfully

Step 8: Query the Newly Created Asset

peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'

Expected Output

{
"ID":"asset10",
"Color":"Black",
"Size":15,
"Owner":"Arun",
"AppraisedValue":800
}

Step 9: Transfer Asset Ownership

peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"TransferAsset","Args":["asset10","David"]}'

Expected Output

Ownership transferred successfully

Step 10: Verify Updated Asset

peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'

Expected Output

{
"ID":"asset10",
"Owner":"David"
}

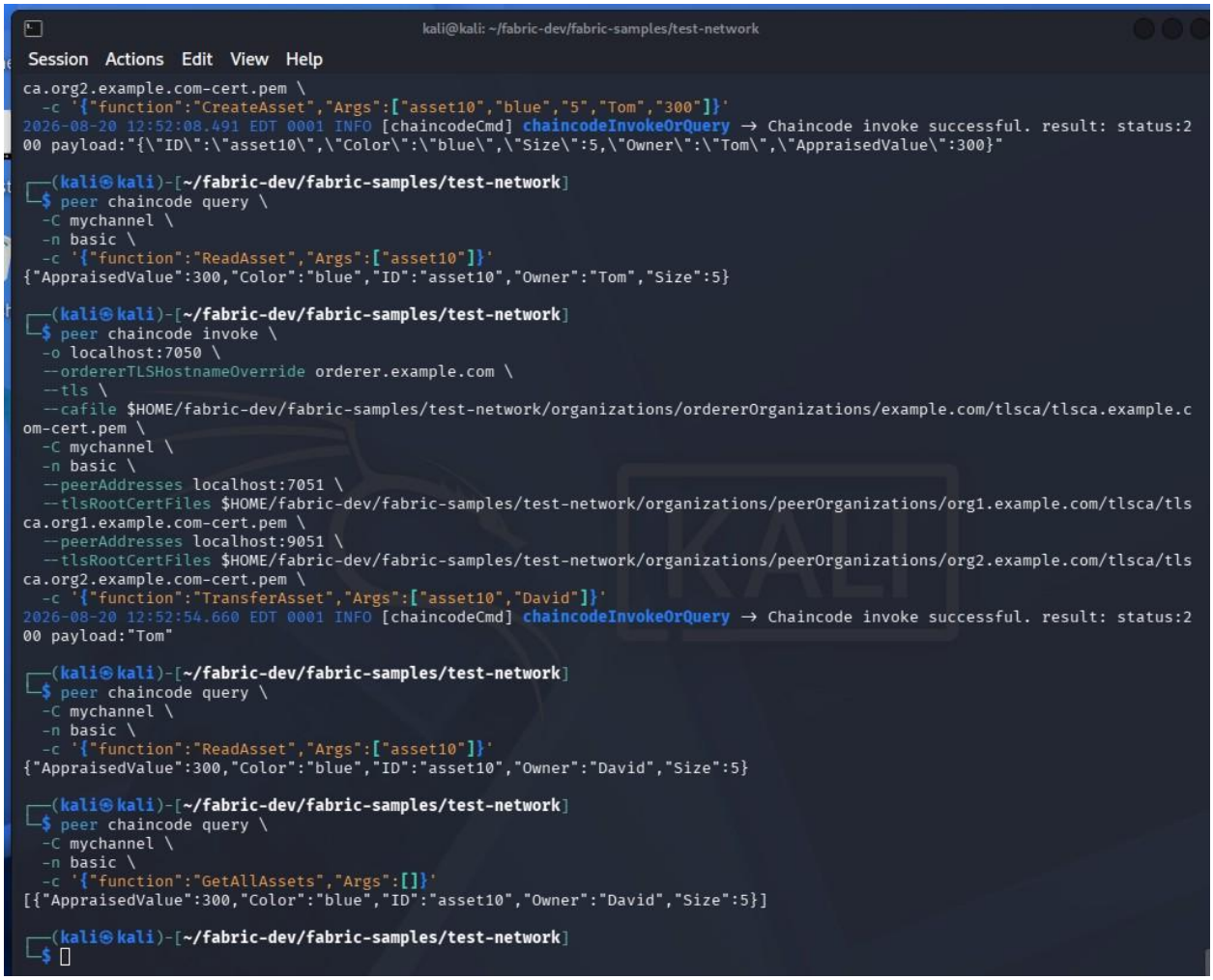

Figure 2.4: Asset creation, transfer and final ledger verification

Step 11: Stop the Network

./network.sh down

Flow of Chaincode Deployment

Start Fabric Network
|
▼
Create Channel
|
▼
Package Chaincode
|
▼
Install Chaincode
|
▼
Approve Chaincode
|
▼
Commit Chaincode
|
▼
Invoke Chaincode
|
▼
Query Ledger
|
▼
Create Asset
|
▼
Transfer Asset
|
▼
Stop Network

Sample Output

Fabric Network Started
Channel Created Successfully
Chaincode Installed Successfully
Chaincode Approved
Chaincode Committed
Ledger Initialized
Asset Created Successfully
Asset Ownership Transferred
Ledger Updated Successfully

Result

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were created, queried, and transferred successfully, demonstrating the basic lifecycle of smart contracts in Hyperledger Fabric.